

Practical Programs

1. Value and Virtual Address

Code

```
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

Output

```
MYNAME = spoorthi
spoorthi@LAPTOP:~$ nano virtualAddress.c
spoorthi@LAPTOP:~$ gcc virtualAddress.c -o virtualAddress
spoorthi@LAPTOP:~$ ./virtualAddress
Value of x = 10
Virtual address of x = 0x7ffc40f699d4
spoorthi@LAPTOP:~$ |
```

2. Code, Data, BSS, Heap, Stack Addresses

Code

```
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {
    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address    = %p\n", (void *)main);
    printf("Data address     = %p\n", (void *)&global);
    printf("BSS address        = %p\n", (void *)&bss_variable);
    printf("Heap address        = %p\n", (void *)heap);
    printf("Stack address       = %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

Output

```
Virtual address of x = 0x7ffe70f1b39d
spoorthi@LAPTOP:~$ nano BSS.c
spoorthi@LAPTOP:~$ gcc BSS.c -o BSS
spoorthi@LAPTOP:~$ ./BSS
Code address = 0x5b68e60a91a9
Data address = 0x5b68e60ac010
BSS address = 0x5b68e60ac018
Heap address = 0x5b6904b61010
Stack address = 0x7ffe70f1b13c
spoorthi@LAPTOP:~$ |
```

3. Page Table

code

```
#include <stdio.h>

int main() {
    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    } else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

Output

```
Stack address = 0x7ffe70f1b13c
spoorthi@LAPTOP:~$ nano pagetable.c
spoorthi@LAPTOP:~$ gcc pagetable.c -o pagetable
spoorthi@LAPTOP:~$ ./pagetable
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
spoorthi@LAPTOP:~$ |
```

4. Virtual Address → Physical Address

Code

```

#include <stdio.h>

int main() {
    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano physicalAddress.c
spoorthi@LAPTOP:~$ gcc physicalAddress.c -o physicalAddress
spoorthi@LAPTOP:~$ ./physicalAddress
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number      = 1
Physical Address = 150
spoorthi@LAPTOP:~$ |

```

5. Dynamic Memory Allocation

Code

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano Memory.c
spoorthi@LAPTOP:~$ gcc Memory.c -o Memory
spoorthi@LAPTOP:~$ ./Memory
10 20 30 40 50
spoorthi@LAPTOP:~$ |

```